23CSE202 DATABASE MANAGEMENT SYSTEMS

S3 B.Tech CSE

Department of Computer Science and Engineering
Amrita School of Computing, Amritapuri Campus

PROJECT REPORT

Group No : C4

Team Members:

Roll No	Student Name
AM.SC.U4CSE25225	KAKI VENKATA PAVITHRA
AM.SC.U4CSE25250	SINGURU CHATHURYA
AM.SC.U4CSE25257	HARSHINI VEDULLA
AM.SC.U4CSE25268	GARA SRIHA

Project Title :

Apex Bank Management System – A Relational Retail Banking Platform

CONTENTS

1. Abstract 2

2. Introduction 3

3. Requirements (Functional & Non-functional) 5

4. UI Design 7

5. Database Design 9

6. Database Selection & Justification 18

7. Implementation 19

8. Testing & Validation 26

9. Results & Screenshots 28

10. Challenges Faced & Solutions 31

11. Conclusion & Future Enhancements 32

References 33

1. Abstract

Banks depend on accurate, consistent records of customers, accounts and money movements, yet many small institutions and student-scale systems still rely on spreadsheets or paper ledgers in which a balance can drift away from the transactions that produced it. The Apex Bank Management System addresses this by placing a carefully designed, normalized relational database at the centre of a retail banking platform. The aim is to link users, branches, accounts, transactions, nominees, debit/credit cards and loans so that every deposit, withdrawal and transfer is recorded as a permanent ledger entry, balances can never become negative, and loans move through a controlled life cycle.

The proposed system uses the relational model with ten relations: User, Customer, Employee, Admin, Branch, Account, Card, Loan, Transaction and Nominee. The User relation is specialised into Customer, Employee and Admin so that role-specific data lives in its own table. Transaction and Nominee are modelled as weak entities owned by Account. The deployed application serves two groups of users through two tabs: customers, who open accounts, move money, register nominees, view their card and apply for loans; and administrators, who audit accounts, loans and users, freeze or close accounts and read summary reports. The schema is implemented in PostgreSQL and is verified to be in Boyce–Codd Normal Form, with primary keys, foreign keys and UNIQUE, CHECK, NOT NULL and DEFAULT constraints enforcing integrity inside the database itself. The design was validated with 28 test cases (12 functional and 16 constraint tests), all of which passed.

2. Introduction

2.1 Problem Domain and Background

Retail banking is one of the most data-intensive services in everyday life. Every day a branch opens accounts, accepts deposits, pays out withdrawals, moves money between accounts, issues cards and processes loan applications. Each of these actions changes stored data, and each must be accurate: a single wrong balance or a lost transaction directly affects a customer's money and the bank's trust.

Banking is a classic example of a transaction-processing problem. A fund transfer, for instance, consists of two linked changes: money leaves one account and arrives in another. If only one of the two changes is stored, money is created or destroyed. The database must therefore guarantee that related changes succeed or fail together, that balances never fall below zero, and that a complete history of every movement is kept for statements and audits.

This is as much a data-management problem as a software problem. The quality of the underlying database design determines whether the platform can keep financial data accurate, consistent and free of redundancy as the number of customers, accounts and transactions grows.

2.2 Existing Scenario and Its Limitations

Many small branches and cooperative institutions still keep customer and account details in ledger books or flat spreadsheets. These approaches have several serious limitations.

The first is redundancy and inconsistency. When customer, account and branch details are repeated on every statement row, a change of phone number or branch address has to be repeated everywhere, and rows that are missed leave the records contradicting each other (update anomalies). Deleting the last transaction of an account may also delete the only copy of the account details (deletion anomaly).

The second is the lack of enforced relationships. In a spreadsheet nothing stops a transaction from referring to an account that does not exist, an account from belonging to no customer, or a loan from pointing to a deleted employee. There is no mechanism that rejects invalid values such as a negative balance, an unknown account type or a nominee share above 100 per cent.

The third is the absence of a reliable audit trail. When ledger entries can be edited or deleted, the history of an account cannot be trusted, and a disputed transaction cannot be reconstructed. Finally, manual systems do not scale: producing branch-wise totals, loan summaries or account statements requires manual effort and is error-prone.

2.3 The Proposed System

The Apex Bank Management System is a retail banking platform designed to remove these limitations through a normalized relational database built in PostgreSQL. The system has two tabs in its web interface, one for customers and one for administrators.

Customers register, sign in and open Savings, Current or Fixed Deposit accounts at a branch. They can deposit, withdraw and transfer funds, view the transaction statement of each account, register nominees with percentage shares, view the card issued for an account and apply for loans of the Personal, Home, Auto or Education type. Administrators sign in to a separate console where they can review users, accounts and loans, freeze or close an account, follow each loan through its status life cycle and view summary reports on transactions, loans and users.

The database consists of ten tables: User, Customer, Employee, Admin, Branch, Account, Card, Loan, Transaction and Nominee, designed up to Boyce–Codd Normal Form. Primary keys, foreign keys and UNIQUE, CHECK, NOT NULL and DEFAULT constraints enforce data integrity at the database level, independent of application logic. Every money movement is written to the Transaction table, which is treated as an append-only ledger: each row stores the balance after the entry, so the history of an account can be replayed at any time. A deposit, withdrawal or transfer updates the ledger and the account balance inside a single database transaction, so either both changes are saved or neither is. The design was verified against 28 test cases, all of which passed.

3. Requirements (Functional & Non-functional)

3.1 Functional requirements

ID	Requirement
FR1	Authentication: users register with name, email, phone and password and sign in with email and password. The role stored in the User table decides whether the Customer or the Admin tab is opened.
FR2	Account opening: a customer opens a Savings, Current or Fixed Deposit account at a chosen branch; the account starts with a zero balance and Active status.
FR3	Account overview: a customer sees all of their accounts with account number, type, branch, balance and status.
FR4	Deposit and withdrawal: a customer deposits money into, or withdraws money from, an account; a withdrawal larger than the balance is rejected.
FR5	Fund transfer: a customer transfers money from one account to another; both ledger entries and both balances change together.
FR6	Statement: a customer views the transaction history of an account with type, amount, date, description and balance after each entry.
FR7	Nominees: a customer registers, views and updates the nominees of an account with name, relationship and percentage share (total 100).
FR8	Card: a customer views the card linked to an account (card type, expiry date and status: Active, Blocked or Expired).
FR9	Loans: a customer applies for a loan (type, principal, interest rate, tenure) and follows its status from Applied to Closed.
FR10	User management (admin): view customers and staff, check KYC status and deactivate a user.
FR11	Account management (admin): view all accounts and freeze, unfreeze or close an account.
FR12	Loan management (admin): review loan applications, assign a processing employee and update the loan status.
FR13	Reports (admin): transaction, loan and user summaries, such as branch-wise balances and loans by status.

3.2 Non-functional requirements

Quality	Requirement
Data integrity	All rules that can be stated as constraints are enforced in the database with primary keys, foreign keys and UNIQUE, CHECK, NOT NULL and DEFAULT constraints.
Reliability	A deposit, withdrawal or transfer is executed as one database transaction (BEGIN, COMMIT, ROLLBACK), so a failure never leaves the ledger and the balance out of step.
Auditability	The Transaction table is an append-only ledger. Rows are never updated or deleted, and each row stores the balance after the entry.
Security	Passwords and card CVV values are stored only as hashes. Roles are separated into Customer, Employee and Admin, and a customer sees only their own accounts.
Consistency	The database is normalized to BCNF so each fact is stored once, which removes update, insertion and deletion anomalies.
Usability	Simple screens: a customer signs in, selects an account and performs an operation in a few steps.
Performance and scalability	Primary-key and UNIQUE constraints create indexes automatically, and the normalized schema allows more branches, customers and accounts to be added without redesign.

Quality	Requirement
Portability	The web interface runs in any modern browser and the database runs on any platform supported by PostgreSQL.

3.3 Business rules and implementation status

The Database Design defined a set of business rules. The table shows how each one is enforced in the final system and in which test it was verified (Section 8).

Business rule	Enforced by	Status
Unique account (email) for every user	UNIQUE constraint on User.Email	Implemented in the database (test T1)
Every user has exactly one role	CHECK on User.Role; subclass tables share UserID with User	Implemented; agreement between Role and the subclass row is application logic
An employee works at exactly one branch	Employee.BranchID is NOT NULL and a foreign key	Implemented (test T13)
Each account has one owner and one branch	Account.CustomerID and Account.BranchID are NOT NULL foreign keys	Implemented (test T3)
Balance never negative	CHECK (Balance >= 0) together with a single database transaction	Implemented (tests F6, T4)
Only valid account types and statuses	CHECK on AccountType and Status	Implemented (test T5)
Ledger is append-only and amounts are positive	CHECK (Amount > 0) and valid TransactionType; the application never updates or deletes ledger rows	Partly implemented; a database trigger is future work (test T9)
Nominee shares between 0 and 100 and totalling 100	CHECK on each row; the total is checked by a query / application logic	Partly implemented (tests T10, F11)
One card per account	Card.AccountID is UNIQUE and NOT NULL; deleting an account deletes its card	Implemented (tests T7, T16)
A loan may exist before a processor is assigned	Loan.EmployeeID is nullable	Implemented (test F9)
Loan status values and default	CHECK on LoanStatus with default Applied; the order of transitions is application logic	Partly implemented (tests T11, F10)
Administrators can freeze or close an account	Account.Status takes the values Active, Frozen or Closed	Implemented (test F8)
A branch cannot be deleted while it is in use	Foreign keys without cascade	Implemented (test T14)

4. UI Design

The Apex Bank Management System presents a web interface with two tabs: a Customer tab for bank customers and an Admin tab for administrators. The interface is deployed on Vercel and reads and writes the PostgreSQL database described in Section 5. The sections below describe what each tab offers and which tables stand behind each screen.

4.1 Design System and Navigation Flow

The interface keeps a simple, consistent layout: a header with the bank name and the two tabs, and a content area that changes with the selected tab and function. Money amounts are shown with two decimal places to match the DECIMAL columns, and status values (Active, Frozen, Closed, Applied, Approved and so on) are shown exactly as stored so the screen and the database always agree.

The customer flow is linear by design: Sign in → Account overview → Account operation (deposit, withdraw, transfer) → Statement, with nominees, card and loans reachable from the account view. The administrator flow is independent: Admin sign in → Overview → Users / Accounts / Loans → Reports. A user who signs in with the Customer role cannot open the Admin tab, because the role is read from the User table.

4.2 Customer Screens

Sign In / Register. A form asks for the email and password of a returning user, and a registration form collects name, email, phone, password and the customer details (address, city and date of birth). A new customer starts with KYC status Pending.

Account overview. Lists every account of the signed-in customer with its account number, type (Savings, Current or Fixed Deposit), branch, balance and status. The customer can open a new account from this screen by choosing the account type and branch.

Deposit, Withdraw and Transfer. The customer selects an account, enters an amount and, for a transfer, the destination account number. The operation succeeds only if the amount is positive and the balance stays zero or above; otherwise an error message is shown and nothing is saved.

Statement. Shows the ledger of one account in order: sequence number, date, type (Deposit, Withdrawal, TransferIn or TransferOut), amount, description and balance after the entry.

Nominees. Lists the nominees of an account with name, relationship and share percentage, and lets the customer add or edit them while the total share stays at 100 per cent.

Card. Shows the card issued for the account: masked card number, type, expiry date and status.

Loans. A form takes the loan type, principal amount, interest rate and tenure in months. A table lists the customer's loans with their current status, so the customer can follow a loan from Applied to Closed.

4.3 Administrator Screens

Admin sign in. The administrator signs in with email and password; the role must be Admin.

Overview. Summary cards give an at-a-glance view of the bank: number of users, accounts, total balance and loans by status.

Users. A table of all users with role, contact details and, for customers, KYC status. The administrator can deactivate a user account.

Accounts. A table of all accounts across branches. The administrator can freeze, unfreeze or close an account by changing its status.

Loans. All loan applications with customer, type, amount, status and assigned employee. The administrator assigns a processing employee and moves the loan through Under Review, Approved, Disbursed and Closed (or Rejected / Defaulted).

Reports. Transaction, loan and user reports, for example branch-wise balances and the number of loans per status, produced with the aggregate queries of Section 7.5.

4.4 Screens and the Tables Behind Them

Screen	Tab	Main tables read or written
Sign in / Register	Both	User, Customer
Account overview, open account	Customer	Account, Branch, Customer
Deposit, Withdraw, Transfer	Customer	Account (balance), Transaction (ledger)
Statement	Customer	Transaction, Account
Nominees	Customer	Nominee, Account
Card	Customer	Card, Account
Loans	Customer	Loan, Customer
Users	Admin	User, Customer, Employee, Admin
Accounts	Admin	Account, Customer, Branch
Loans (management)	Admin	Loan, Employee, User
Reports	Admin	Account, Branch, Transaction, Loan

5. Database Design

5.1 Entities and attributes

The ER model has ten entities. USER is a superclass specialised into CUSTOMER, EMPLOYEE and ADMIN, and TRANSACTION and NOMINEE are weak entities owned by ACCOUNT. Primary keys are marked (PK).

Entity	Description	Attributes
USER	Every person who can log in (superclass).	UserID (PK), Name, Email, Phone, Password, Role, CreatedAt
CUSTOMER	Subclass of USER for bank customers.	UserID (PK, FK), Address, City, DateOfBirth, KYCStatus
EMPLOYEE	Subclass of USER for branch staff.	UserID (PK, FK), BranchID, Designation, Salary
ADMIN	Subclass of USER for administrators.	UserID (PK, FK), AccessLevel
BRANCH	A physical bank branch.	BranchID (PK), BranchName, IFSCCode, Address, City, ContactNumber, IsActive
ACCOUNT	A bank account opened by one customer at one branch.	AccountID (PK), AccountNumber, CustomerID, BranchID, AccountType, Balance, OpenedDate, Status
TRANSACTION	Weak entity: one ledger entry of an account.	AccountID + TransactionSeqNo (PK), TransactionType, Amount, TransactionDate, BalanceAfter, Description
NOMINEE	Weak entity: a beneficiary of an account.	AccountID + NomineeSeqNo (PK), NomineeName, Relationship, SharePercentage
CARD	The card issued for one account.	CardID (PK), AccountID, CardNumber, CardType, ExpiryDate, CVV, Status
LOAN	A loan applied for by a customer and optionally processed by an employee.	LoanID (PK), CustomerID, EmployeeID, LoanType, PrincipalAmount, InterestRate, TenureMonths, LoanStatus, DisbursedDate

5.2 Relationships and cardinalities

Relationship	Entities	Cardinality	Participation	Meaning
is_a (Role)	USER → CUSTOMER / EMPLOYEE / ADMIN	1 : 0..1 each	Disjoint, total	Every user is exactly one of the three roles.
works_at	EMPLOYEE – BRANCH	M : 1	Total (Employee) / Partial (Branch)	An employee works at one branch; a branch has many employees.
opens	CUSTOMER – ACCOUNT	1 : M	Total (Account) / Partial (Customer)	A customer may own many accounts; each account has one owner.
opened_at	ACCOUNT – BRANCH	M : 1	Total (Account) / Partial (Branch)	Each account is opened at one branch.
has_txn (identifying)	ACCOUNT – TRANSACTION	1 : M	Total (Transaction) / Partial (Account)	An account has many ledger entries; an entry belongs to one account.
has_nominee (identifying)	ACCOUNT – NOMINEE	1 : M	Total (Nominee) / Partial (Account)	An account may have many nominees.
linked_to	ACCOUNT – CARD	1 : 1	Total (Card) / Partial (Account)	A card belongs to exactly one account and an account has at most one card.

Relationship	Entities	Cardinality	Participation	Meaning
applies_for	CUSTOMER – LOAN	1 : M	Total (Loan) / Partial (Customer)	A customer may apply for many loans; each loan belongs to one customer.
processed_by	EMPLOYEE – LOAN	1 : M	Partial (both sides)	A loan may be assigned to one employee, who may process many loans.

5.3 ER diagram

Chen notation is used: rectangles are entities (double border = weak entity), diamonds are relationships (double diamond = identifying relationship) and the circle marked d denotes the disjoint specialisation of USER. Only entity names are shown in the diagram; the attributes of every entity are listed in Section 5.1 and Section 5.5. The labels 1 and M give the cardinality of each relationship.

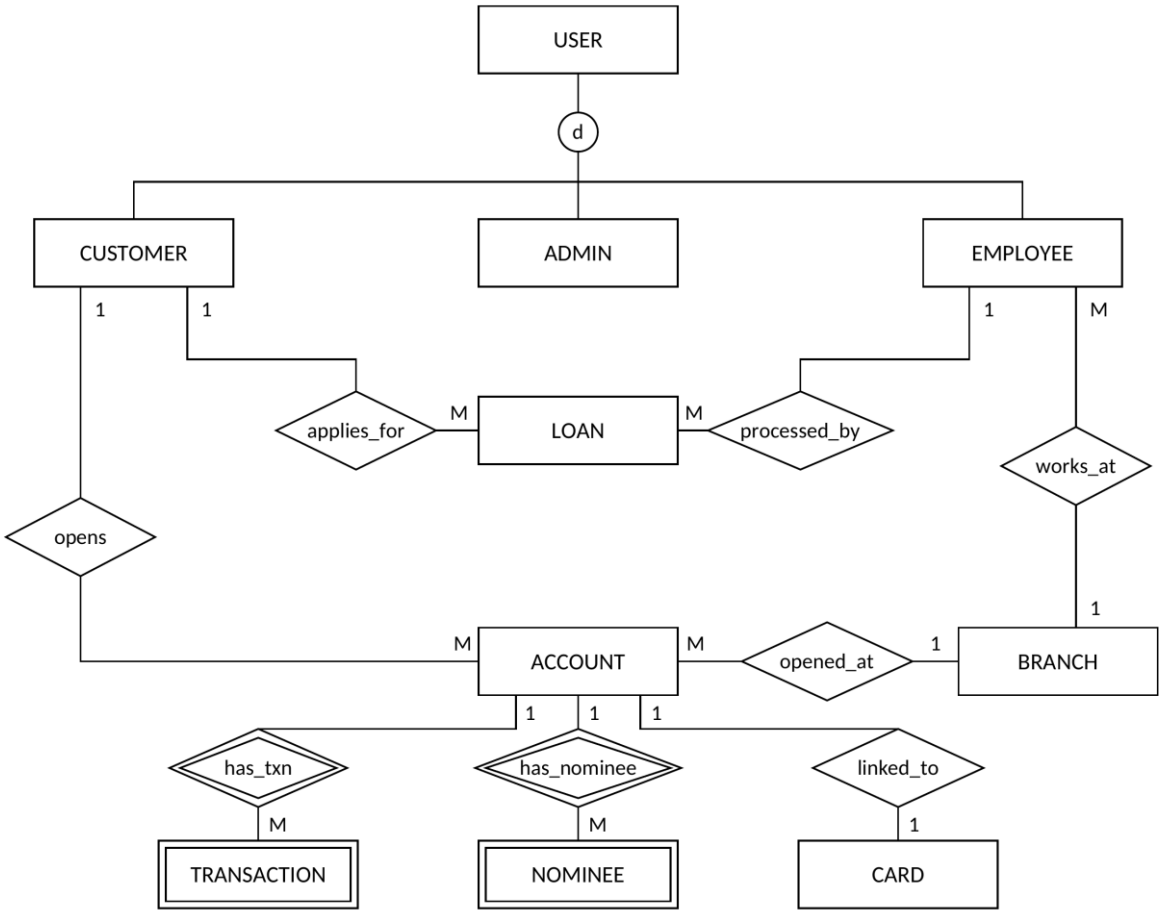


Figure 1: Entity Relationship (Chen) diagram of the Apex Bank Management System database

5.4 Conversion of the ER model into relational tables

- Each strong entity becomes a table and its key attribute becomes the primary key (a SERIAL integer, which PostgreSQL fills automatically): BRANCH, ACCOUNT, CARD and LOAN.

- The disjoint, total specialisation of USER is implemented with one table per subclass. CUSTOMER, EMPLOYEE and ADMIN each use UserID as both primary key and foreign key referencing USER, with ON DELETE CASCADE.
- Each weak entity (TRANSACTION, NOMINEE) becomes a table whose primary key is the composite of the owner key and its own discriminator: (AccountID, TransactionSeqNo) and (AccountID, NomineeSeqNo).
- Each 1 : M relationship is implemented by placing the primary key of the "1" side as a foreign key on the "M" side: Employee.BranchID, Account.CustomerID, Account.BranchID, Loan.CustomerID and Loan.EmployeeID (nullable because the processor is optional).
- The 1 : 1 relationship between Account and Card is implemented with a UNIQUE, NOT NULL foreign key Card.AccountID; deleting an account deletes its card (ON DELETE CASCADE).
- Derived value: Account.Balance is a maintained summary of the ledger, and Transaction.BalanceAfter stores the balance at the time of each entry.
- All foreign keys other than the cascading ones use the default RESTRICT action, so a branch, customer or account that is still referenced cannot be deleted by mistake.

5.5 Relational schema

PK = primary key, FK = foreign key. The table "User" is written with quotation marks in SQL because USER is a reserved word in PostgreSQL.

1. User Table

Column Name	Data Type	Constraints	Description
UserID	SERIAL	PK	Unique identifier of the user
Name	VARCHAR(80)	NOT NULL	Full name
Email	VARCHAR(100)	UNIQUE, NOT NULL	Unique login email
Phone	VARCHAR(15)	NOT NULL	Contact phone number
Password	VARCHAR(255)	NOT NULL	Hashed password
Role	VARCHAR(20)	NOT NULL, CHECK IN (Customer, Employee, Admin)	Role of the user
CreatedAt	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Registration time

2. Customer Table

Column Name	Data Type	Constraints	Description
UserID	INT	PK, FK (User), ON DELETE CASCADE	Same value as User.UserID
Address	VARCHAR(150)		Residential address
City	VARCHAR(50)		City
DateOfBirth	DATE		Date of birth

Column Name	Data Type	Constraints	Description
KYCStatus	VARCHAR(20)	DEFAULT 'Pending', CHECK IN (Pending, Verified, Rejected)	KYC verification state

3. Employee Table

Column Name	Data Type	Constraints	Description
UserID	INT	PK, FK (User), ON DELETE CASCADE	Same value as User.UserID
BranchID	INT	FK (Branch), NOT NULL	Branch where the employee works
Designation	VARCHAR(40)		Job title
Salary	DECIMAL(10,2)	CHECK (Salary > 0)	Salary

4. Admin Table

Column Name	Data Type	Constraints	Description
UserID	INT	PK, FK (User), ON DELETE CASCADE	Same value as User.UserID
AccessLevel	VARCHAR(20)		Level of administrative access

5. Branch Table

Column Name	Data Type	Constraints	Description
BranchID	SERIAL	PK	Unique identifier of the branch
BranchName	VARCHAR(100)	NOT NULL	Branch name
IFSCCode	VARCHAR(11)	UNIQUE, NOT NULL	IFSC code of the branch
Address	VARCHAR(150)	NOT NULL	Branch address
City	VARCHAR(50)	NOT NULL	City
ContactNumber	VARCHAR(15)		Branch phone number
IsActive	BOOLEAN	DEFAULT TRUE	Whether the branch is operating

6. Account Table

Column Name	Data Type	Constraints	Description
AccountID	SERIAL	PK	Unique identifier of the account
AccountNumber	VARCHAR(18)	UNIQUE, NOT NULL	Account number shown to the customer
CustomerID	INT	FK (Customer), NOT NULL	Owner of the account
BranchID	INT	FK (Branch), NOT NULL	Branch where it was opened
AccountType	VARCHAR(20)	NOT NULL, CHECK IN (Savings, Current, Fixed Deposit)	Type of account
Balance	DECIMAL(12,2)	DEFAULT 0, CHECK (Balance >= 0)	Current balance
OpenedDate	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Opening time

Column Name	Data Type	Constraints	Description
Status	VARCHAR(20)	DEFAULT 'Active', CHECK IN (Active, Frozen, Closed)	State of the account

7. Card Table

Column Name	Data Type	Constraints	Description
CardID	SERIAL	PK	Unique identifier of the card
AccountID	INT	FK (Account), UNIQUE, NOT NULL, ON DELETE CASCADE	Account the card belongs to
CardNumber	VARCHAR(16)	UNIQUE, NOT NULL	Card number
CardType	VARCHAR(20)	NOT NULL, CHECK IN (Debit, Credit)	Type of card
ExpiryDate	DATE	NOT NULL	Expiry date
CVV	VARCHAR(255)	NOT NULL	Hashed CVV
Status	VARCHAR(20)	DEFAULT 'Active', CHECK IN (Active, Blocked, Expired)	State of the card

8. Loan Table

Column Name	Data Type	Constraints	Description
LoanID	SERIAL	PK	Unique identifier of the loan
CustomerID	INT	FK (Customer), NOT NULL	Customer who applied
EmployeeID	INT	FK (Employee), nullable	Employee processing the loan (assigned later)
LoanType	VARCHAR(30)	NOT NULL, CHECK IN (Personal, Home, Auto, Education)	Type of loan
PrincipalAmount	DECIMAL(12,2)	CHECK (PrincipalAmount > 0)	Loan amount
InterestRate	DECIMAL(4,2)	CHECK (InterestRate > 0)	Annual interest rate (%)
TenureMonths	INT	CHECK (TenureMonths > 0)	Repayment period in months
LoanStatus	VARCHAR(20)	DEFAULT 'Applied', CHECK IN (Applied, UnderReview, Approved, Rejected, Disbursed, Closed, Defaulted)	Stage of the loan
DisbursedDate	TIMESTAMP		Set when the loan is disbursed

9. Transaction Table (weak entity)

Column Name	Data Type	Constraints	Description
AccountID	INT	PK, FK (Account)	Owner account
TransactionSeqNo	INT	PK (discriminator)	Sequence number within the account

Column Name	Data Type	Constraints	Description
TransactionType	VARCHAR(20)	NOT NULL, CHECK IN (Deposit, Withdrawal, TransferIn, TransferOut)	Kind of entry
Amount	DECIMAL(12,2)	CHECK (Amount > 0)	Amount moved
TransactionDate	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Time of the entry
BalanceAfter	DECIMAL(12,2)	NOT NULL	Balance after this entry
Description	VARCHAR(255)		Narration

10. Nominee Table (weak entity)

Column Name	Data Type	Constraints	Description
AccountID	INT	PK, FK (Account)	Owner account
NomineeSeqNo	INT	PK (discriminator)	Sequence number within the account
NomineeName	VARCHAR(80)	NOT NULL	Name of the nominee
Relationship	VARCHAR(30)		Relationship to the account holder
SharePercentage	DECIMAL(5,2)	CHECK (SharePercentage >= 0 AND SharePercentage <= 100)	Share of the nominee (%)

5.6 Functional dependencies

FD	Dependency	Significance
FD1	UserID → Name, Email, Phone, Password, Role, CreatedAt	Every user detail is tied to one identifier, so profile data is stored once.
FD2	UserID → Address, City, DateOfBirth, KYCStatus (Customer)	Customer-specific data depends only on the user identifier.
FD3	UserID → BranchID, Designation, Salary (Employee)	An employee belongs to exactly one branch.
FD4	UserID → AccessLevel (Admin)	Administrative rights depend only on the user.
FD5	BranchID → BranchName, IFSCCode, Address, City, ContactNumber, IsActive	Branch details are stored once and referenced by BranchID.
FD6	AccountID → AccountNumber, CustomerID, BranchID, AccountType, Balance, OpenedDate, Status	An account determines its owner, branch and state.
FD7	CardID → AccountID, CardNumber, CardType, ExpiryDate, CVV, Status	Card details depend on the card identifier.
FD8	LoanID → CustomerID, EmployeeID, LoanType, PrincipalAmount, InterestRate, TenureMonths, LoanStatus, DisbursedDate	A loan determines its customer, optional processor and terms.
FD9	{AccountID, TransactionSeqNo} → TransactionType, Amount, TransactionDate, BalanceAfter, Description	A ledger entry is identified by its account and sequence number.
FD10	{AccountID, NomineeSeqNo} → NomineeName, Relationship, SharePercentage	A nominee is identified by its account and sequence number.
FD11	Email → UserID	Email is unique, so it is a candidate key of USER.
FD12	AccountNumber → AccountID	Account number is unique, so it is a candidate key of ACCOUNT.
FD13	IFSCCode → BranchID	IFSC code is unique, so it is a candidate key of BRANCH.

FD	Dependency	Significance
FD14	CardNumber → CardID; AccountID → CardID	Card number and the 1:1 account link are candidate keys of CARD.

5.7 Normalization

To show the normalization steps we start from one large unnormalized (UNF) relation that an unnormalized design would use to print account statements. It holds the account, customer, branch and transaction facts together, with the transactions as a repeating group:

ACCOUNT_STATEMENT (AccountID, AccountNumber, AccountType, CustomerID, CustomerName, CustomerPhone, BranchID, BranchName, IFSCCode, BranchCity, { TransactionSeqNo, TransactionType, Amount, TransactionDate, BalanceAfter, Description })

This structure repeats customer and branch information on every row and causes redundancy and update, insertion and deletion anomalies.

1NF — First Normal Form

The repeating group of transactions is removed by giving each transaction its own row and repeating the account facts on every row. All attributes are now atomic and the key becomes composite:

STATEMENT_REPORT (AccountID, TransactionSeqNo, TransactionType, Amount, TransactionDate, BalanceAfter, Description, AccountNumber, AccountType, CustomerID, CustomerName, CustomerPhone, BranchID, BranchName, IFSCCode, BranchCity) — 16 attributes.

Candidate key by attribute closure

The dependencies of this relation are FD1 to FD6 of the table below (taken from FD9, FD6, FD1, FD5, FD12 and FD13 above).

Label	Dependency
A	{AccountID, TransactionSeqNo} → TransactionType, Amount, TransactionDate, BalanceAfter, Description
B	AccountID → AccountNumber, AccountType, CustomerID, BranchID
C	CustomerID → CustomerName, CustomerPhone
D	BranchID → BranchName, IFSCCode, BranchCity
E	AccountNumber → AccountID
F	IFSCCode → BranchID

Closure of {AccountID, TransactionSeqNo}: start with { AccountID, TransactionSeqNo }; apply A to add TransactionType, Amount, TransactionDate, BalanceAfter and Description; apply B to add AccountNumber, AccountType, CustomerID and BranchID; apply C to add CustomerName and CustomerPhone; apply D to add BranchName, IFSCCode and BranchCity. The closure contains all 16 attributes, so {AccountID, TransactionSeqNo} is a super key. Neither AccountID alone nor TransactionSeqNo alone reaches every attribute, so the key is

minimal: candidate key 1 = {AccountID, TransactionSeqNo}. Because of E, candidate key 2 = {AccountNumber, TransactionSeqNo} is also minimal. Candidate key 1 is used as the primary key.

2NF — Second Normal Form

A relation is in 2NF when no non-prime attribute depends on only part of a candidate key. Here dependency B (AccountID → AccountNumber, AccountType, CustomerID, BranchID) depends on only part of the key, and the customer and branch attributes also depend on AccountID alone through CustomerID and BranchID. STATEMENT_REPORT therefore violates 2NF: the account facts are repeated on every transaction row, so changing a customer's phone number means editing every transaction. The partial dependencies are removed by decomposition:

- TRANSACTION_2NF (AccountID, TransactionSeqNo, TransactionType, Amount, TransactionDate, BalanceAfter, Description)
- ACCOUNT_2NF (AccountID, AccountNumber, AccountType, CustomerID, CustomerName, CustomerPhone, BranchID, BranchName, IFSCCode, BranchCity)

3NF — Third Normal Form

TRANSACTION_2NF is already in 3NF. ACCOUNT_2NF still contains two transitive dependencies: AccountID → CustomerID → CustomerName, CustomerPhone, and AccountID → BranchID → BranchName, IFSCCode, BranchCity. CustomerID and BranchID are non-key attributes that determine other non-key attributes, so ACCOUNT_2NF violates 3NF. It is split into:

- ACCOUNT_3NF (AccountID, AccountNumber, AccountType, CustomerID, BranchID)
- CUSTOMER_3NF (CustomerID, CustomerName, CustomerPhone)
- BRANCH_3NF (BranchID, BranchName, IFSCCode, BranchCity)

together with TRANSACTION_3NF = TRANSACTION_2NF. All four relations are in 3NF, and the decomposition is lossless and dependency-preserving.

BCNF — Boyce–Codd Normal Form

A relation is in BCNF when the determinant of every non-trivial dependency is a super key.

Relation	Non-trivial dependencies	Determinant is a super key?
TRANSACTION_3NF	{AccountID, TransactionSeqNo} → all other attributes	Yes
ACCOUNT_3NF	AccountID → rest; AccountNumber → AccountID and rest	Yes (both are candidate keys)
CUSTOMER_3NF	CustomerID → CustomerName, CustomerPhone	Yes
BRANCH_3NF	BranchID → rest; IFSCCode → BranchID and rest	Yes (both are candidate keys)

Mapping to the final schema. TRANSACTION_3NF becomes TRANSACTION (same key {AccountID, TransactionSeqNo}, modelled as a weak entity); ACCOUNT_3NF becomes

ACCOUNT (extended with Balance, OpenedDate and Status); CUSTOMER_3NF becomes USER together with CUSTOMER (CustomerName and CustomerPhone are stored once in USER as Name and Phone); BRANCH_3NF becomes BRANCH (BranchCity is BRANCH.City). EMPLOYEE, ADMIN, CARD, NOMINEE and LOAN were not part of the statement report; they were added at the ER stage as separate relations, each keyed on its own identifier, which avoids repeating card, nominee or loan data across transaction rows.

Summary of the normal forms

Normal form	What was done	Result
1NF	The repeating group of transactions inside ACCOUNT_STATEMENT was flattened into one row per transaction; every attribute is atomic.	No repeating groups.
2NF	Account, customer and branch facts that depended only on AccountID were moved out of the statement relation into ACCOUNT_2NF.	No partial dependencies.
3NF	The transitive dependencies through CustomerID and BranchID were removed by creating CUSTOMER_3NF and BRANCH_3NF.	No non-key attribute depends on another non-key attribute.
BCNF	In every relation each determinant is a candidate key, including the alternate keys Email, AccountNumber, IFSCCode, CardNumber and Card.AccountID.	Every determinant is a super key.
4NF	The independent multi-valued facts of an account (ledger entries, nominees, card) are already stored in separate relations, so no relation has a non-trivial multivalued dependency.	No multivalued dependencies.
5NF	The relations are joined only through primary and foreign keys, so no join dependency exists that is not implied by the candidate keys.	No further decomposition needed.

BCNF verification of all 10 relations in the final schema

Relation	Candidate keys	Non-trivial FDs	BCNF
USER	UserID, Email	UserID → all; Email → all	Yes
CUSTOMER	UserID	UserID → Address, City, DateOfBirth, KYCStatus	Yes
EMPLOYEE	UserID	UserID → BranchID, Designation, Salary	Yes
ADMIN	UserID	UserID → AccessLevel	Yes
BRANCH	BranchID, IFSCCode	each key → all	Yes
ACCOUNT	AccountID, AccountNumber	each key → all	Yes
CARD	CardID, AccountID, CardNumber	each key → all	Yes
LOAN	LoanID	LoanID → all	Yes
TRANSACTION	{AccountID, TransactionSeqNo}	key → all	Yes
NOMINEE	{AccountID, NomineeSeqNo}	key → all	Yes

The final decomposition is the ten relations USER, CUSTOMER, EMPLOYEE, ADMIN, BRANCH, ACCOUNT, CARD, LOAN, TRANSACTION and NOMINEE described in Section 5.5. Two pieces of data are redundant by design: Account.Balance, which can be

recomputed from the ledger, and Transaction.BalanceAfter, a snapshot of the running balance. Neither is determined by a non-key attribute, so they are not normal-form violations; the consistency query Q4 in Section 7.5 checks that they agree.

6. Database Selection & Justification

The project uses PostgreSQL, an open-source object-relational database system. The same SQL (SERIAL keys, CHECK constraints, ON DELETE CASCADE, ON CONFLICT and DECIMAL types) is used in the schema and the sample data.

6.1 Why PostgreSQL is suitable

- **ACID transactions:** a deposit, withdrawal or transfer inserts ledger rows and updates balances. PostgreSQL commits all of these changes or none, so the ledger and the balances cannot disagree (tests F4 to F7).
- **Rich constraints:** PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, NOT NULL and DEFAULT let the database itself reject invalid data such as a negative balance, a duplicate email or a nominee share above 100 (tests T1 to T16).
- **Exact numeric types:** DECIMAL(12,2) stores money without rounding errors, which floating-point types cannot guarantee.
- **Referential actions:** ON DELETE CASCADE removes subclass rows and cards together with their parent, while the default RESTRICT protects branches, customers and accounts that are still in use.
- **Powerful SQL:** joins, LEFT JOIN, GROUP BY / HAVING and aggregate functions are used for statements and reports (Section 7.5).
- **Procedural support:** PL/pgSQL triggers can enforce the ledger and loan-status rules inside the database, which is listed as future work.
- **Cost and tooling:** free and open source, with detailed documentation and tools such as pgAdmin; it runs on Windows, Linux and macOS.

6.2 Comparison with other options

Feature	PostgreSQL	MySQL	Oracle
Licence / cost	Free, open source	Community edition is free	Commercial licence for full features
ACID transactions	Yes	Yes (InnoDB engine)	Yes
CHECK constraints	Yes	Yes (from version 8.0.16)	Yes
Exact DECIMAL type	Yes	Yes	Yes (NUMBER)
Procedural language	PL/pgSQL	Stored procedures	PL/SQL
Ease of setup for a student project	Easy	Easy	Heavier installation

All three systems could be used, but PostgreSQL combines strict standards-compliant SQL, strong transaction support and zero cost, which makes it the best fit for a banking design in which data correctness matters most.

7. Implementation

7.1 Architecture and technologies

The Apex Bank Management System follows a three-tier architecture. The web interface (presentation tier) sends requests to the application tier, which runs SQL on PostgreSQL (data tier). The browser never talks to the database directly.

Tier	Technology	Role
Presentation	Web interface with Customer and Admin tabs, deployed on Vercel (https://dbms-project-m3ui.vercel.app/)	Screens, forms and tables for customers and administrators
Application	Node (express framework)	Authentication, validation, business logic and database transactions
Data	PostgreSQL (verified on version 16.2)	Tables, constraints, queries and transactions

7.2 Database creation and table creation

The ten tables are created with the DDL below. Parent tables are created before the tables that reference them, and the script begins with DROP TABLE IF EXISTS ... CASCADE so that it can be re-run on a clean database. The Loan table allows a NULL EmployeeID because a loan is assigned to an employee only after it has been applied for.

```
CREATE TABLE Branch (
    BranchID SERIAL PRIMARY KEY,
    BranchName VARCHAR(100) NOT NULL,
    IFSCCode VARCHAR(11) UNIQUE NOT NULL,
    Address VARCHAR(150) NOT NULL,
    City VARCHAR(50) NOT NULL,
    ContactNumber VARCHAR(15),
    IsActive BOOLEAN DEFAULT TRUE
);

CREATE TABLE "User" (
    UserID SERIAL PRIMARY KEY,
    Name VARCHAR(80) NOT NULL,
    Email VARCHAR(100) UNIQUE NOT NULL,
    Phone VARCHAR(15) NOT NULL,
    Password VARCHAR(255) NOT NULL,
    Role VARCHAR(20) CHECK (Role IN ('Customer', 'Employee', 'Admin')) NOT NULL,
    CreatedAt TIMESTAMP DEFAULT CURRENT  TIMESTAMP
);

CREATE TABLE Customer (
    UserID INT PRIMARY KEY,
    Address VARCHAR(150),
    City VARCHAR(50),
    DateOfBirth DATE,
    KYCStatus VARCHAR(20) DEFAULT 'Pending' CHECK (KYCStatus IN ('Pending', 'Verified', 'Rejected')),
    FOREIGN KEY (UserID) REFERENCES "User"(UserID) ON DELETE CASCADE
);

CREATE TABLE Employee (
    UserID INT PRIMARY KEY,
    BranchID INT NOT NULL,
    Designation VARCHAR(40),
    Salary DECIMAL(10,2) CHECK (Salary > 0),
    FOREIGN KEY (UserID) REFERENCES "User"(UserID) ON DELETE CASCADE,
    FOREIGN KEY (BranchID) REFERENCES Branch(BranchID)
);
```

```
CREATE TABLE Admin (
    UserID INT PRIMARY KEY,
    AccessLevel VARCHAR(20),
    FOREIGN KEY (UserID) REFERENCES "User"(UserID) ON DELETE CASCADE
);

CREATE TABLE Account (
    AccountID SERIAL PRIMARY KEY,
    AccountNumber VARCHAR(18) UNIQUE NOT NULL,
    CustomerID INT NOT NULL,
    BranchID INT NOT NULL,
    AccountType VARCHAR(20) CHECK (AccountType IN ('Savings', 'Current', 'Fixed Deposit')) NOT NULL,
    Balance DECIMAL(12,2) DEFAULT 0 CHECK (Balance >= 0),
    OpenedDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    Status VARCHAR(20) DEFAULT 'Active' CHECK (Status IN ('Active', 'Frozen', 'Closed')),
    FOREIGN KEY (CustomerID) REFERENCES Customer(UserID),
    FOREIGN KEY (BranchID) REFERENCES Branch(BranchID)
);

CREATE TABLE Card (
    CardID SERIAL PRIMARY KEY,
    AccountID INT UNIQUE NOT NULL,
    CardNumber VARCHAR(16) UNIQUE NOT NULL,
    CardType VARCHAR(20) CHECK (CardType IN ('Debit', 'Credit')) NOT NULL,
    ExpiryDate DATE NOT NULL,
    CVV VARCHAR(255) NOT NULL,
    Status VARCHAR(20) DEFAULT 'Active' CHECK (Status IN ('Active', 'Blocked', 'Expired')),
    FOREIGN KEY (AccountID) REFERENCES Account(AccountID) ON DELETE CASCADE
);

CREATE TABLE Loan (
    LoanID SERIAL PRIMARY KEY,
    CustomerID INT NOT NULL,
    EmployeeID INT,
    LoanType VARCHAR(30) CHECK (LoanType IN ('Personal', 'Home', 'Auto', 'Education')) NOT NULL,
    PrincipalAmount DECIMAL(12,2) CHECK (PrincipalAmount > 0),
    InterestRate DECIMAL(4,2) CHECK (InterestRate > 0),
    TenureMonths INT CHECK (TenureMonths > 0),
    LoanStatus VARCHAR(20) DEFAULT 'Applied' CHECK (LoanStatus IN ('Applied', 'UnderReview', 'Approved', 'Rejected',
'Disbursed', 'Closed', 'Defaulted')),
    DisbursedDate TIMESTAMP,
    FOREIGN KEY (CustomerID) REFERENCES Customer(UserID),
    FOREIGN KEY (EmployeeID) REFERENCES Employee(UserID)
);

CREATE TABLE Transaction (
    AccountID INT NOT NULL,
    TransactionSeqNo INT NOT NULL,
    TransactionType VARCHAR(20) CHECK (TransactionType IN ('Deposit', 'Withdrawal', 'TransferIn', 'TransferOut')) NOT
NULL,
    Amount DECIMAL(12,2) CHECK (Amount > 0),
    TransactionDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    BalanceAfter DECIMAL(12,2) NOT NULL,
    Description VARCHAR(255),
    PRIMARY KEY (AccountID, TransactionSeqNo),
    FOREIGN KEY (AccountID) REFERENCES Account(AccountID)
);

CREATE TABLE Nominee (
    AccountID INT NOT NULL,
    NomineeSeqNo INT NOT NULL,
    NomineeName VARCHAR(80) NOT NULL,
    Relationship VARCHAR(30),
    SharePercentage DECIMAL(5,2) CHECK (SharePercentage >= 0 AND SharePercentage <= 100),
    PRIMARY KEY (AccountID, NomineeSeqNo),
    FOREIGN KEY (AccountID) REFERENCES Account(AccountID)
);
```

7.3 Sample data

The tables were loaded with sample data: 2 branches, 5 users (3 customers, 1 employee and 1 administrator), 3 accounts, 2 cards and 3 nominees. The passwords in the sample are placeholders, not real hashes. All queries and tests in this report were run on this data.

```
INSERT INTO Branch (BranchName,IFSCCode,Address,City,ContactNumber) VALUES
 ('Apex Main Branch','APEX0000001','Main Financial Hub','Mumbai','022-12345678'),
 ('Apex Kochi Branch','APEX0000002','MG Road','Kochi','0484-1234567');
INSERT INTO "User" (Name,Email,Phone,Password,Role) VALUES
 ('Aarav Nair','aarav@example.com','9876500001','scrypt$sample1','Customer'),
 ('Meera Menon','meera@example.com','9876500002','scrypt$sample2','Customer'),
 ('Rohan Das','rohan@example.com','9876500003','scrypt$sample3','Customer'),
 ('Kiran Rao','kiran@example.com','9876500004','scrypt$sample4','Employee'),
 ('Admin User','admin@example.com','9876500005','scrypt$sample5','Admin');
INSERT INTO Customer (UserID,Address,City,DateOfBirth) VALUES (1,'12 Lake Road','Kochi','2001-04-12'),(2,
  '7 Hill View','Mumbai','1999-09-30'),(3,'3 Park Street','Kochi','2000-01-18');
UPDATE Customer SET KYCStatus='Verified' WHERE UserID IN (1,2);
INSERT INTO Employee (UserID,BranchID,Designation,Salary) VALUES (4,1,'Loan Officer',45000);
INSERT INTO Admin (UserID,AccessLevel) VALUES (5,'Full');
INSERT INTO Account (AccountNumber,CustomerID,BranchID,AccountType) VALUES
 ('APX000000000001',1,1,'Savings'),('APX000000000002',2,1,'Current'),('APX000000000003',1,2,'Fixed Deposit');
INSERT INTO Card (AccountID,CardNumber,CardType,ExpiryDate,CVV) VALUES (1,'4000123412341234','Debit',
  '2030-06-30','hash$cvv1'),(2,'4000123412345678','Debit','2030-06-30','hash$cvv2');
INSERT INTO Nominee (AccountID,NomineeSeqNo,NomineeName,Relationship,SharePercentage) VALUES (1,1,'Anita Nair',
  'Mother',60),(1,2,'Arun Nair','Father',40),(2,1,'Rita Menon','Spouse',50);
```

7.4 Data operations and SQL

Operation	SQL operation	Used by
Register	INSERT into User, then INSERT into Customer (KYCStatus defaults to Pending)	Customer
Sign in	SELECT from User by Email; Role selects the Customer or Admin tab	Customer / Admin
Open account	INSERT into Account (Balance 0, Status Active)	Customer
Issue card	INSERT into Card (one per account)	Customer / Admin
Deposit / Withdraw	Transaction: INSERT ledger row into Transaction, UPDATE Account.Balance	Customer
Transfer	Transaction: two ledger rows (TransferOut, TransferIn) and two balance updates	Customer
Statement	SELECT from Transaction ORDER BY TransactionSeqNo	Customer / Admin
Nominees	INSERT / UPDATE / SELECT on Nominee	Customer
Apply for loan	INSERT into Loan (LoanStatus defaults to Applied, EmployeeID NULL)	Customer
Process loan	UPDATE Loan SET EmployeeID, LoanStatus, DisbursedDate	Admin
Freeze / close account	UPDATE Account SET Status	Admin
Reports	JOIN and GROUP BY queries over Account, Branch, Transaction and Loan	Admin

7.5 Important SQL queries and their outputs

This section shows the main SQL queries used in the Apex Bank database, one at a time, in simple steps. Each query was run on PostgreSQL 16 with the sample data of Section 7.3, and the output below is the real result.

Q1. Show all customers

A simple SELECT shows every customer with the city and the KYC status.

```
SELECT UserID, Address, City, KYCStatus
FROM Customer ORDER BY UserID;
```

Output:

userid	address	city	kycstatus
1	12 Lake Road	Kochi	Verified
2	7 Hill View	Mumbai	Verified
6	5 Beach Road	Kochi	Pending

Q2. Show all users and their roles

Every person who can log in is stored in the User table. The word User is written inside double quotes because USER is a reserved word in PostgreSQL; without the quotes, SELECT * FROM User returns the name of the database login instead of the table. The Password column is left out on purpose.

```
SELECT UserID, Name, Email, Role
FROM "User" ORDER BY UserID;
```

Output:

userid	name	email	role
1	Aarav Nair	aarav@example.com	Customer
2	Meera Menon	meera@example.com	Customer
4	Kiran Rao	kiran@example.com	Employee
5	Admin User	admin@example.com	Admin
6	Neha Iyer	neha@example.com	Customer

Q3. Allow a loan before an employee is chosen

A customer applies for a loan before any employee is known, so EmployeeID must be allowed to stay empty (NULL). This query removes the NOT NULL rule from that column.

```
ALTER TABLE Loan ALTER COLUMN EmployeeID DROP NOT NULL;
```

Output:

ALTER TABLE

Q4. Add the default branch only once

The main branch is added with ON CONFLICT DO NOTHING. If a branch with the same IFSC code already exists, nothing is added and no error is raised, so the query can be run again safely.

```
INSERT INTO Branch (BranchName, Address, City, IFSCCode)
VALUES ('Apex Main Branch','Main Financial Hub','Mumbai','APEX0000001')
ON CONFLICT (IFSCCode) DO NOTHING;


SELECT BranchID, BranchName, City, IFSCCode FROM Branch;
```

Output:

INSERT 0 0 (no row added, because the branch already exists). The second query then shows the two branches:

branchid	branchname	city	ifsccode
1	Apex Main Branch	Mumbai	APEX0000001
2	Apex Kochi Branch	Kochi	APEX0000002

Q5. Show each account with its owner and branch

This joins the Account, User and Branch tables so that the account number is shown together with the owner name and the branch name.

```
SELECT a.AccountNumber, u.Name AS Owner, b.BranchName,
    a.AccountType, a.Balance, a.Status
FROM Account a
JOIN "User" u ON u.UserID = a.CustomerID
JOIN Branch b ON b.BranchID = a.BranchID
ORDER BY a.AccountID;
```

Output:

accountnumber	owner	branchname	accounttype	balance	status
APX000000000001	Aarav Nair	Apex Main Branch	Savings	0.00	Active
APX000000000002	Meera Menon	Apex Main Branch	Current	1000.00	Active
APX000000000003	Aarav Nair	Apex Kochi Branch	Fixed Deposit	0.00	Active
APX000000000004	Neha Iyer	Apex Main Branch	Savings	2000.00	Frozen

Q6. Total money in each branch

GROUP BY puts the accounts of each branch together; COUNT gives the number of accounts and SUM gives the total balance.

```
SELECT b.BranchName, COUNT(a.AccountID) AS Accounts,
    SUM(a.Balance) AS TotalBalance
FROM Branch b JOIN Account a ON a.BranchID = b.BranchID
GROUP BY b.BranchName
HAVING COUNT(a.AccountID) >= 1
ORDER BY TotalBalance DESC;
```

Output:

branchname	accounts	totalbalance
Apex Main Branch	3	3000.00
Apex Kochi Branch	1	0.00

Q7. Show loans with the employee who handled them

A LEFT JOIN keeps a loan in the result even when no employee is assigned yet; in that case the ProcessedBy column is empty (NULL).

```
SELECT l.LoanID, cu.Name AS Customer, l.LoanType,
    l.PrincipalAmount, l.LoanStatus, e.Name AS ProcessedBy
FROM Loan l
JOIN "User" cu ON cu.UserID = l.CustomerID
```

```
LEFT JOIN "User" e ON e.UserID = l.EmployeeID;
```

Output:

loanid	customer	loantype	principalamount	loanstatus	processedby
1	Aarav Nair	Personal	200000.00	Disbursed	Kiran Rao

Q8. Account statement

The statement of one account is its list of transactions, in order. Here it is shown for account 4.

```
SELECT TransactionSeqNo AS Seq, TransactionType, Amount, BalanceAfter
FROM Transaction WHERE AccountID = 4
ORDER BY TransactionSeqNo;
```

Output:

seq	transactiontype	amount	balanceafter
1	Deposit	5000.00	5000.00
2	Withdrawal	2000.00	3000.00
3	TransferOut	1000.00	2000.00

7.6 Security and integrity measures

- Passwords and card CVV values are stored only as hashes (VARCHAR(255)); the plain values are never stored.
- The Role column is restricted by a CHECK constraint, and each role has its own table, so role-specific data cannot leak into other roles.
- CHECK constraints reject negative balances, non-positive amounts and invalid statuses, and foreign keys reject references to rows that do not exist.
- The default RESTRICT action on branch, customer and account references stops accidental deletion of data that is still in use; only subclass rows and cards cascade.
- An account can be frozen or closed by an administrator through its Status column without deleting any history.

8. Testing & Validation

The design was tested on a PostgreSQL 16.2 database created with the DDL of Section 7.2 and loaded with the sample data of Section 7.3. Functional tests run the same SQL operations that the application performs (deposits, transfers, loan updates and so on) and then check the stored result. Constraint tests insert invalid rows directly into the database. Every test case below was run and the actual output is the real response of PostgreSQL.

8.1 Functional test cases

Each test below does one simple thing, such as opening an account or making a deposit, and then checks the result stored in the database. All twelve tests passed.

ID	What is tested	Input	Expected result	Actual result	Result
F1	Register a new customer	New user with Role = Customer	Customer saved; KYCStatus is Pending	2 rows inserted, UserID = 6, KYCStatus = Pending	Pass
F2	Open a savings account	CustomerID, BranchID, type = Savings	New account with Balance 0 and Status Active	AccountID = 4, Balance = 0.00, Status = Active	Pass
F3	Issue a debit card	AccountID, CardType = Debit	Card saved with Status Active	Card inserted, Status = Active	Pass
F4	Deposit 5000	Amount = 5000	COMMIT; Balance = 5000	COMMIT; Balance = 5000.00	Pass
F5	Withdraw 2000	Amount = 2000	COMMIT; Balance = 3000	COMMIT; Balance = 3000.00	Pass
F6	Withdraw more than the balance	Amount = 10000, balance = 3000	Rejected; nothing is saved	new row for relation "account" violates check constraint "account_balance_check" (ROLLBACK); Balance still 3000.00; ledger rows = 2	Pass
F7	Transfer 1000 from account 4 to account 2	Amount = 1000	COMMIT; sender 2000, receiver 1000	COMMIT; sender balance = 2000.00, receiver balance = 1000.00	Pass
F8	Admin freezes an account	Status = Frozen	Status changes to Frozen	Status = Frozen	Pass
F9	Apply for a loan before an employee is chosen	Personal, 200000, 10.50%, 24 months	Loan saved with LoanStatus Applied and EmployeeID empty	LoanID = 1, LoanStatus = Applied, EmployeeID = NULL	Pass
F10	Employee approves and disburses the loan	EmployeeID = 4	Employee set; status Disbursed; DisbursedDate filled	Approved by employee 4, then Disbursed; DisbursedDate set = true	Pass
F11	Find accounts whose nominee shares are not 100	Query on the Nominee table	Account 2 (only 50) is listed; account 1 (60 + 40) is not	Listed: AccountID 2 = 50.00	Pass
F12	Delete a user who has no accounts	DELETE User 3	User and Customer rows are both removed	DELETE 1; Customer rows for UserID 3 = 0	Pass

8.2 Constraint and data integrity tests

ID	Test case	Input	Expected output	Actual output (PostgreSQL)	Result
T1	UNIQUE: duplicate user email	Email = aarav@example.com again	unique violation	duplicate key value violates unique constraint "User_email_key"	Pass
T2	CHECK: invalid role	Role = Manager	check violation	new row for relation "User" violates check constraint "User_role_check"	Pass

ID	Test case	Input	Expected output	Actual output (PostgreSQL)	Result
T3	FOREIGN KEY: account for a customer that does not exist	CustomerID = 999	foreign key violation	insert or update on table "account" violates foreign key constraint "account_customerid_fkey"	Pass
T4	CHECK: negative balance	Balance = -100	check violation	new row for relation "account" violates check constraint "account_balance_check"	Pass
T5	CHECK: invalid account type	AccountType = Salary	check violation	new row for relation "account" violates check constraint "account_accounttype_check"	Pass
T6	UNIQUE: duplicate account number	AccountNumber = APX000000000001	unique violation	duplicate key value violates unique constraint "account_accountnumber_key"	Pass
T7	UNIQUE: second card for the same account (1:1)	AccountID = 1 again	unique violation	duplicate key value violates unique constraint "card_accountid_key"	Pass
T8	PRIMARY KEY: duplicate ledger entry	AccountID 4, TransactionSeqNo 1 again	primary key violation	duplicate key value violates unique constraint "transaction_pkey"	Pass
T9	CHECK: transaction amount of zero	Amount = 0	check violation	new row for relation "transaction" violates check constraint "transaction_amount_check"	Pass
T10	CHECK: nominee share above 100	SharePercentage = 120	check violation	new row for relation "nominee" violates check constraint "nominee_sharepercentage_check"	Pass
T11	CHECK: invalid loan status	LoanStatus = Pending	check violation	new row for relation "loan" violates check constraint "loan_loanstatus_check"	Pass
T12	CHECK: loan principal not positive	PrincipalAmount = -5	check violation	new row for relation "loan" violates check constraint "loan_principalamount_check"	Pass
T13	NOT NULL: employee without a branch	BranchID = NULL	not-null violation	null value in column "branchid" of relation "employee" violates not-null constraint	Pass
T14	RESTRICT: delete a branch that still has accounts	DELETE Branch 1	foreign key violation (no cascade)	update or delete on table "branch" violates foreign key constraint "employee_branchid_fkey" on table "employee"	Pass
T15	UNIQUE: duplicate IFSC code	IFSCCode = APEX0000001	unique violation	duplicate key value violates unique constraint "branch_ifsccode_key"	Pass
T16	ON DELETE CASCADE: deleting an account removes its card	DELETE Account 2 (ledger and nominee rows removed first)	Card row removed	DELETE 1; Card rows for account 2 = 0	Pass

All 28 test cases passed: 12 functional and 16 constraint tests.

9. Results & Screenshots

The screenshots below show the main screens of the deployed application (https://dbms-project-m3ui.vercel.app/). The query results and test outputs of the database layer are given in Sections 7.5 and 8.

9.1 Customer tab

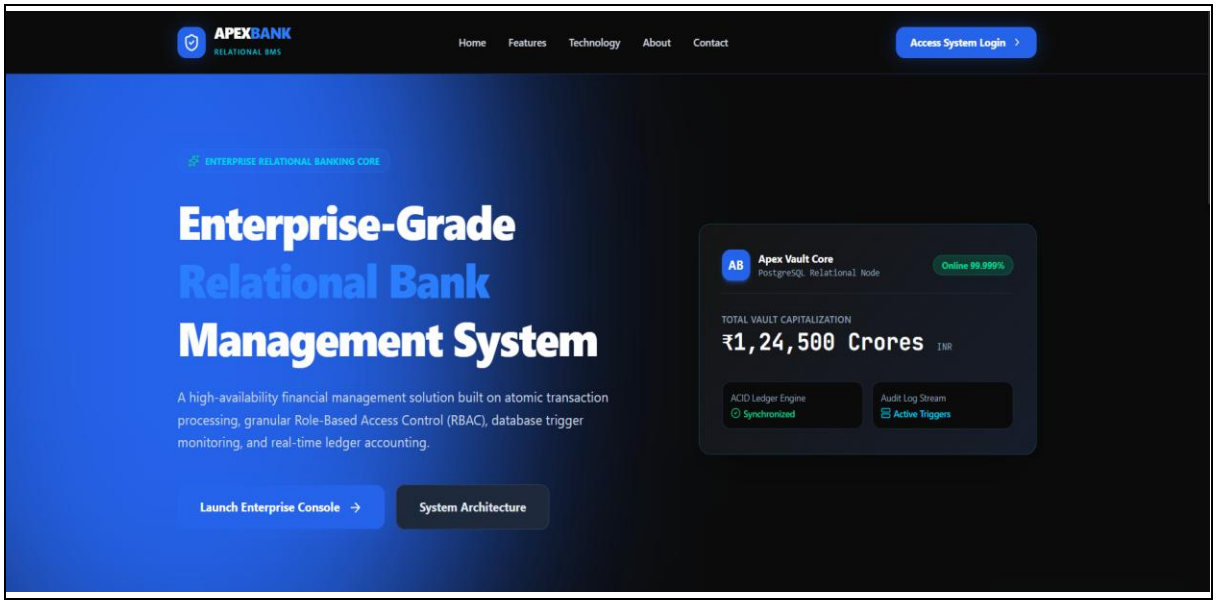


Figure 1: Main Dashboard

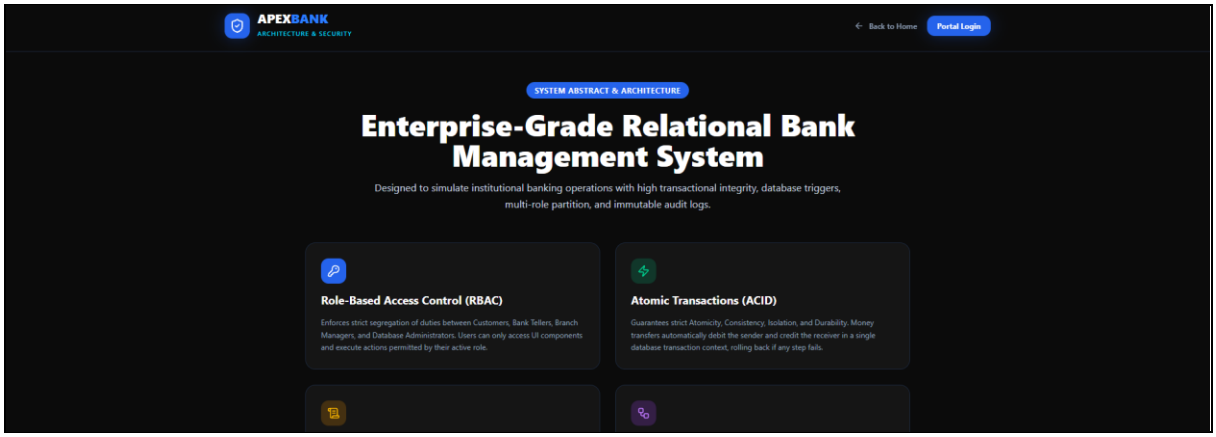


Figure 2:System architechure page

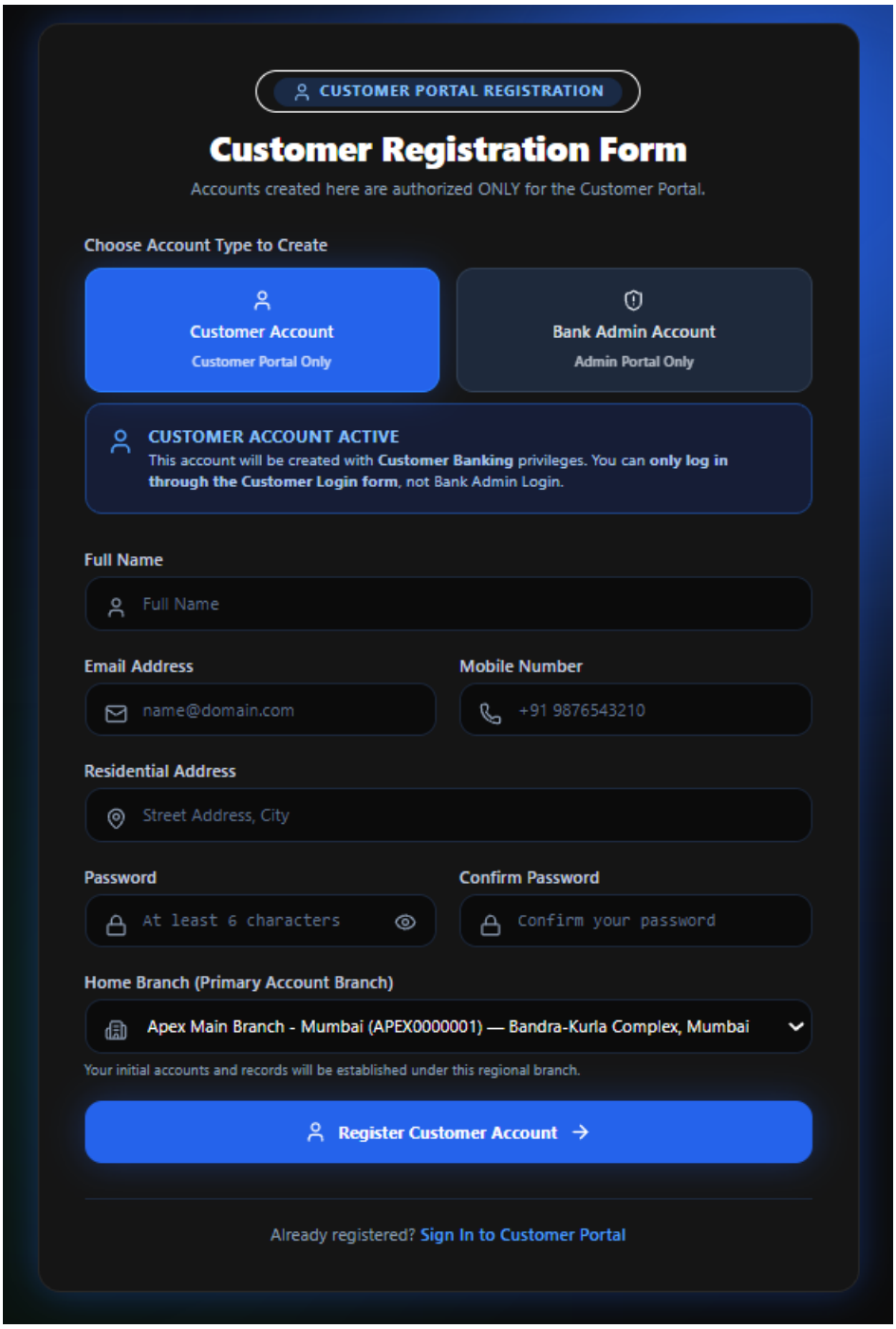


Figure 3:Account register page

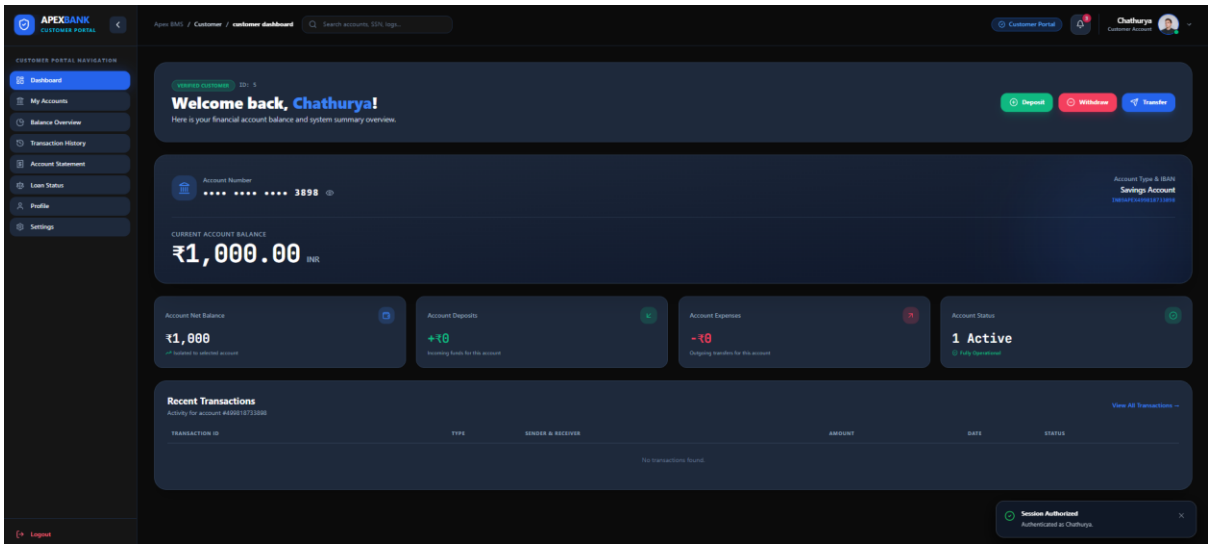


Figure 4: Main Customer dashboard

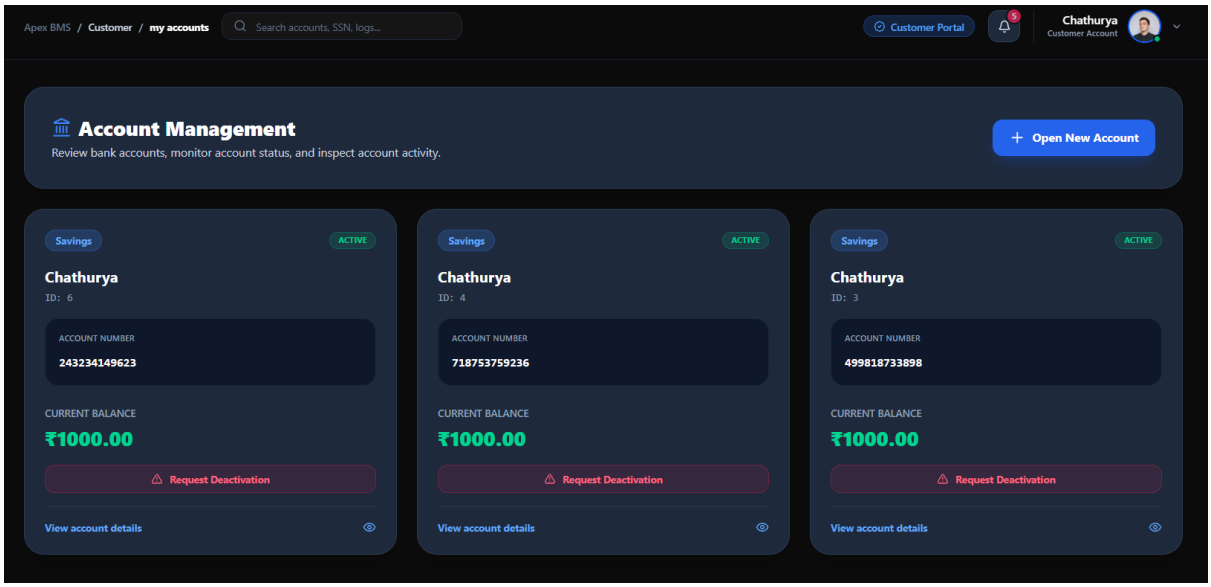


Figure 5: Accounts page

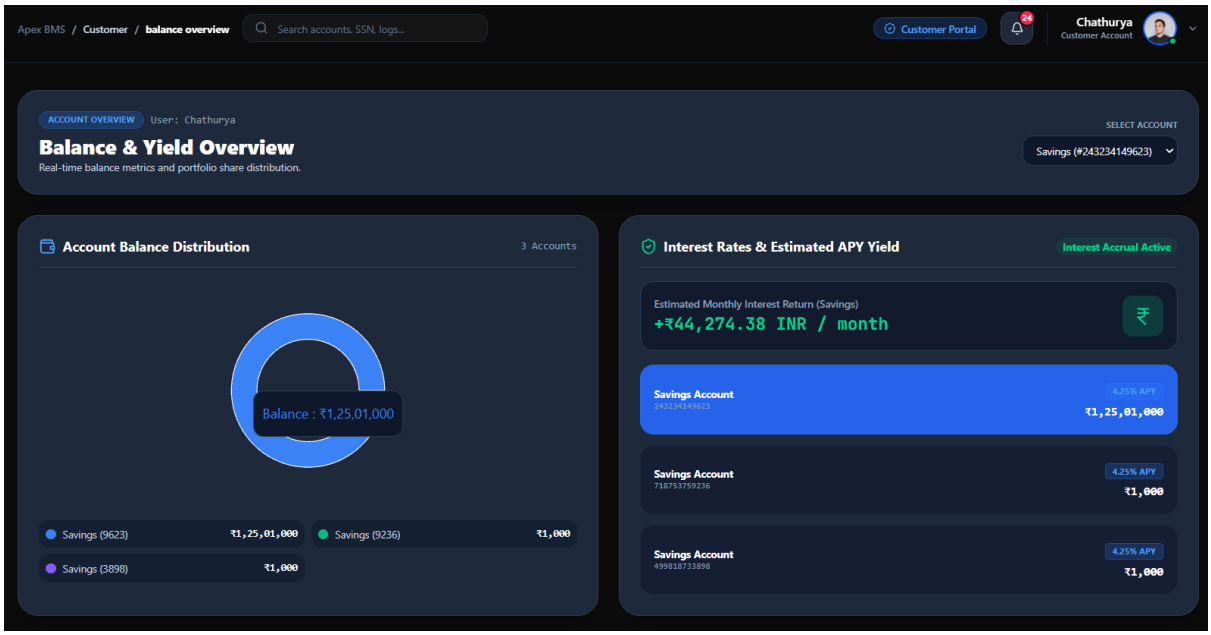


Figure 6: Balance overview

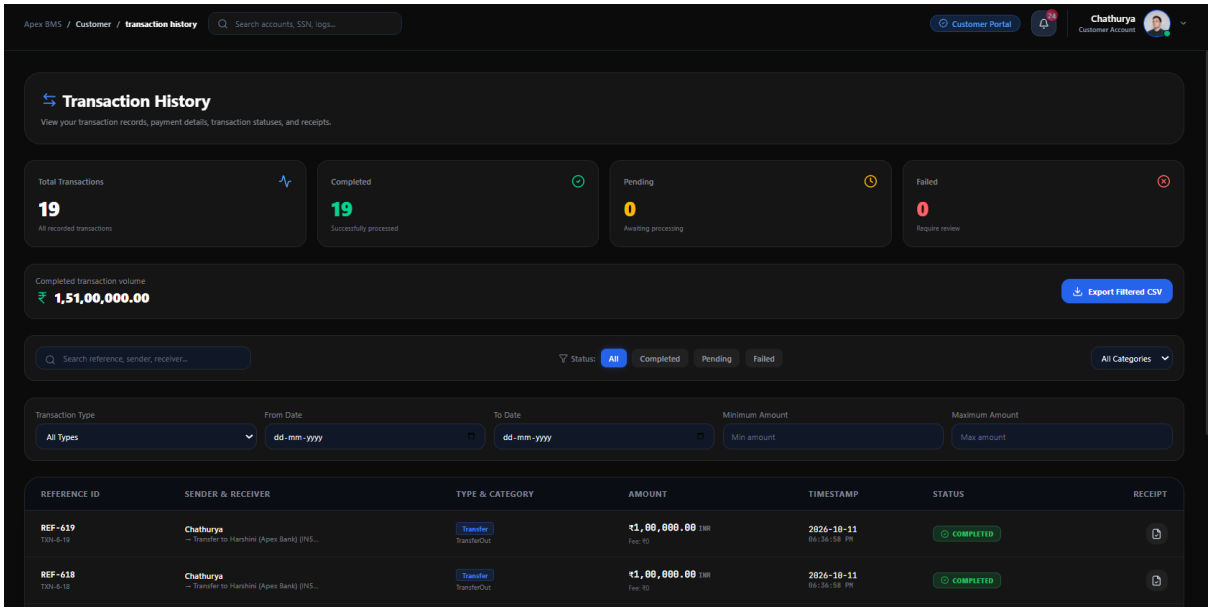


Figure 7: Transaction history

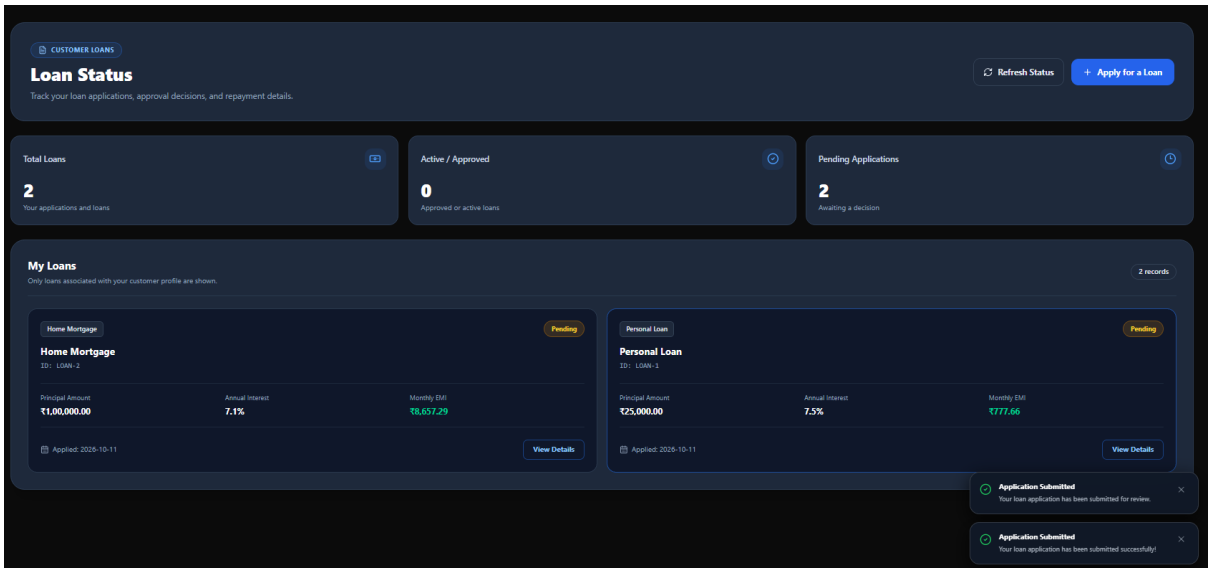


Figure 8: Loan management

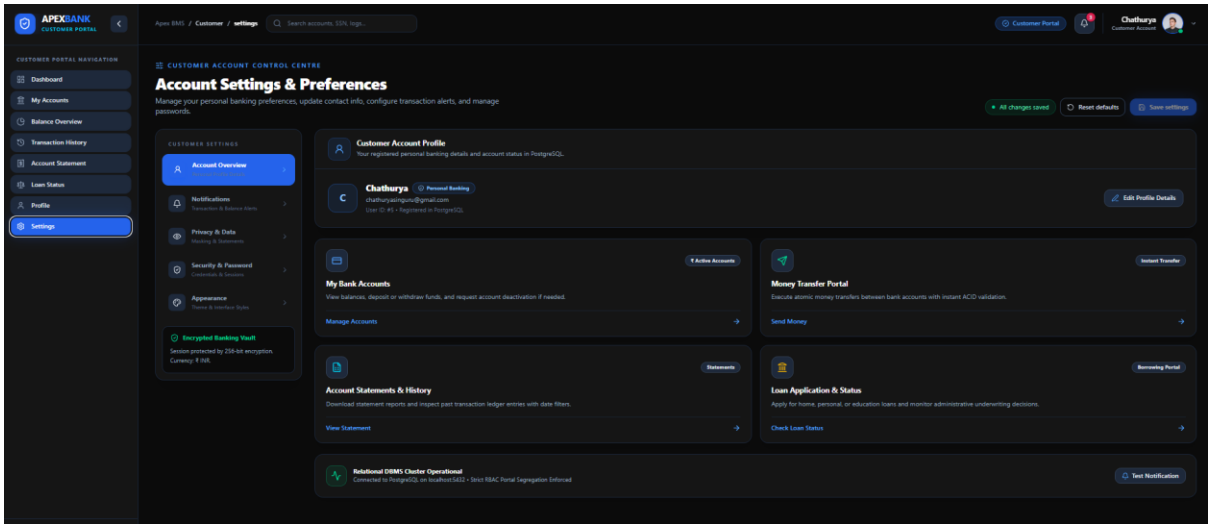


Figure 9: Settings

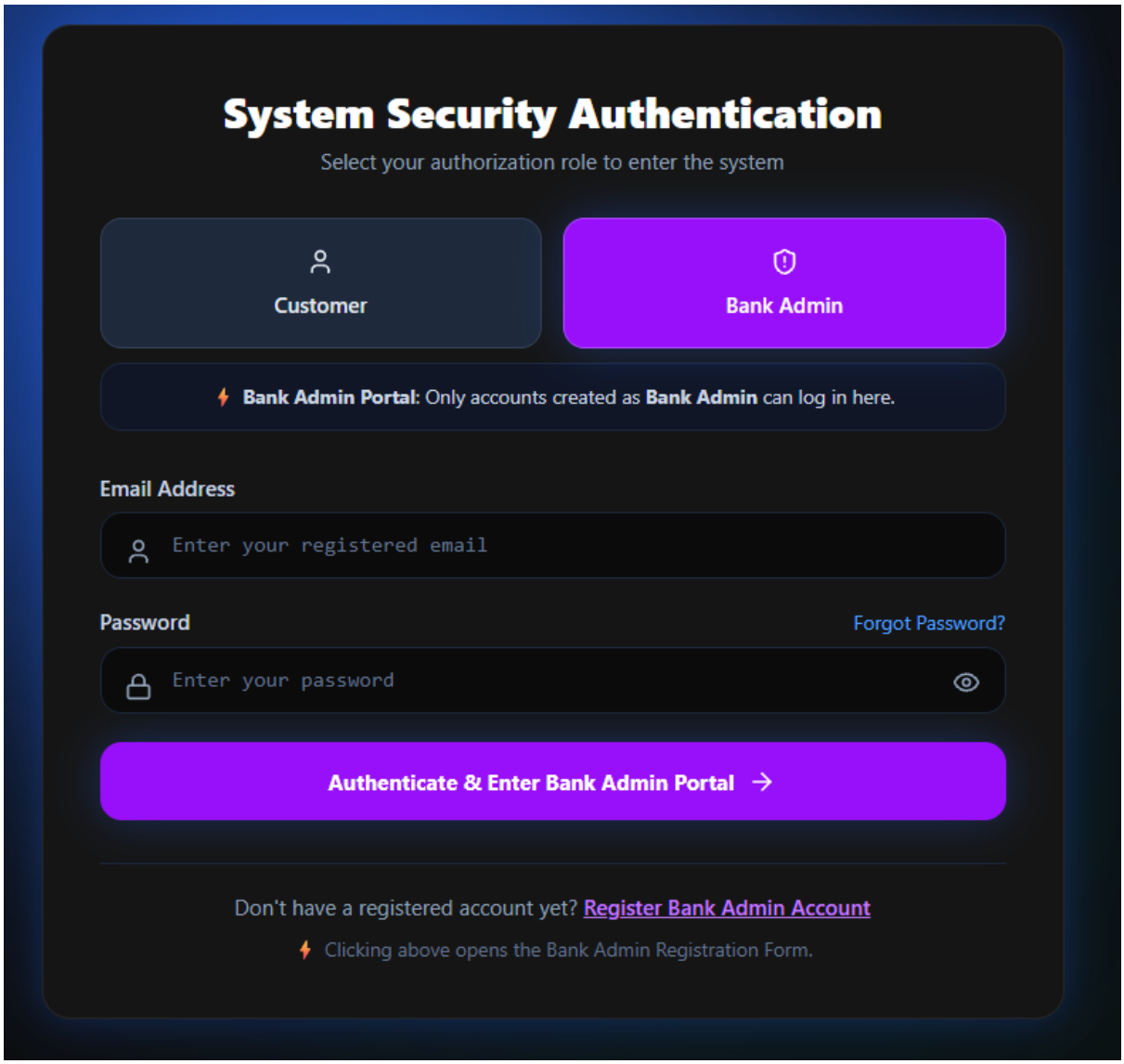


Figure 10: Back to admin register page

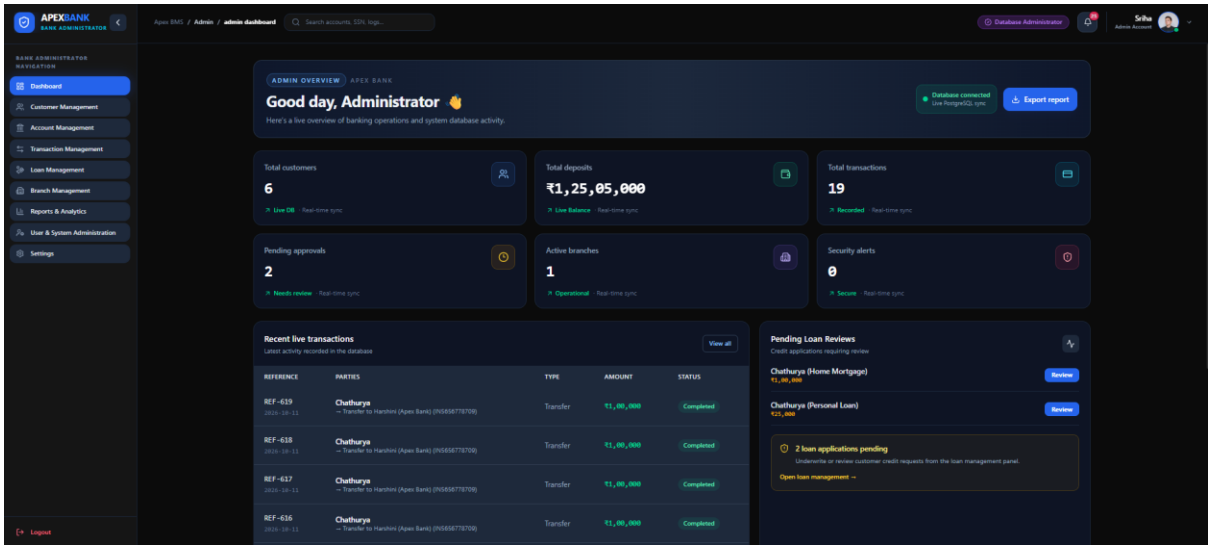


Figure 11: Admin dashboard

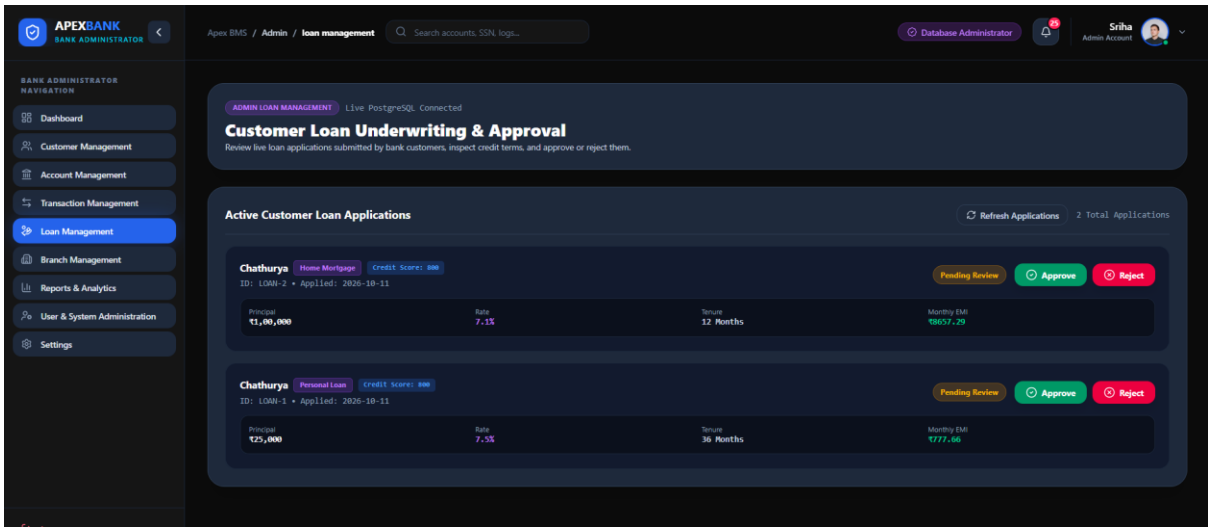


Figure 12: Loan approval page

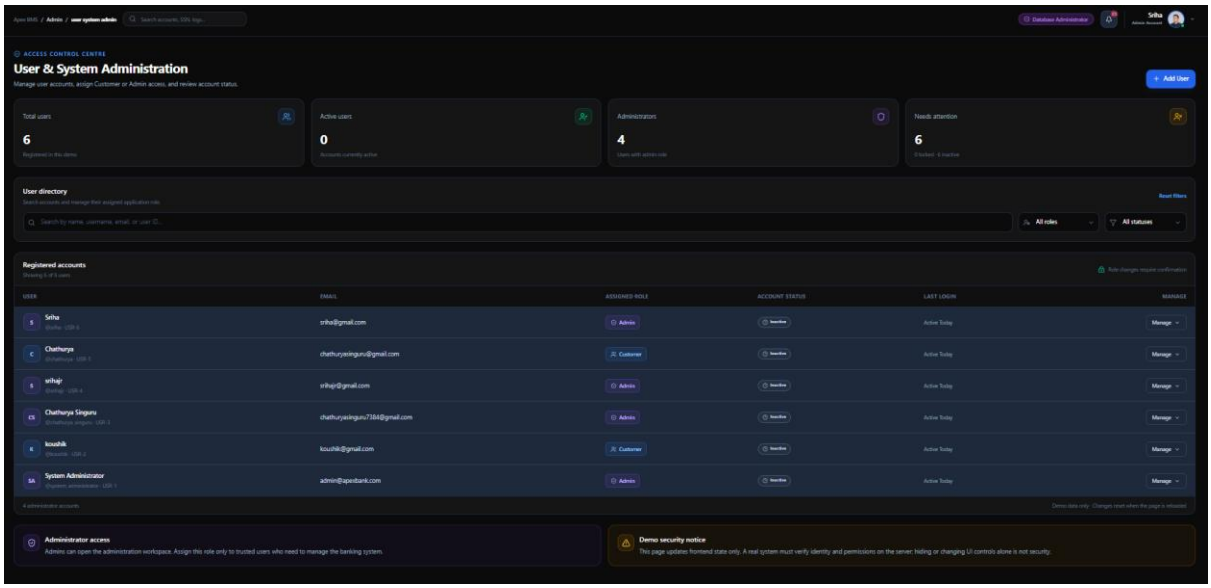


Figure 13: User and system administration

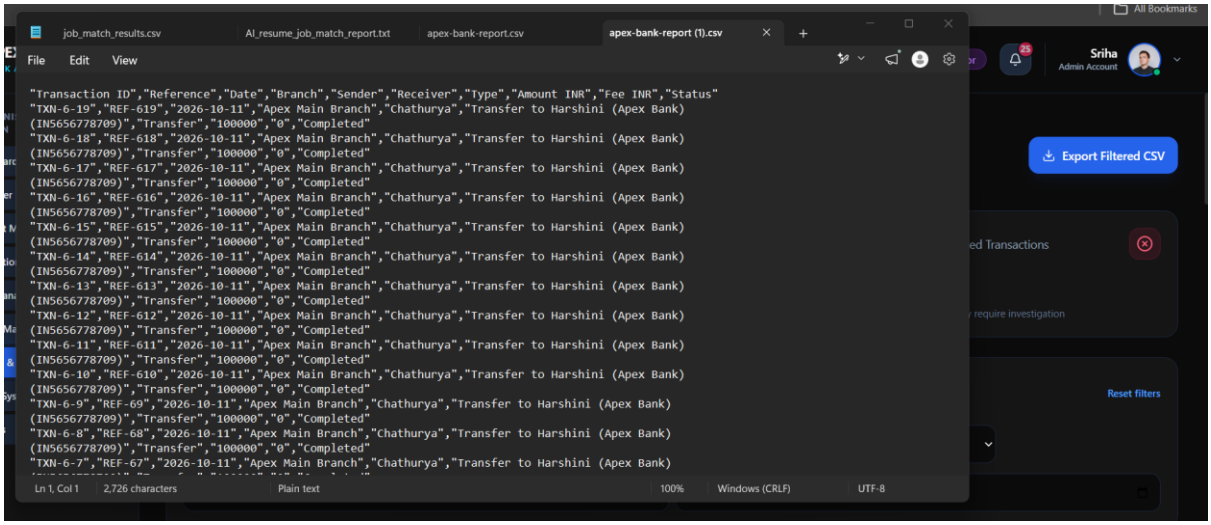


Figure 14: Exported accounts statement

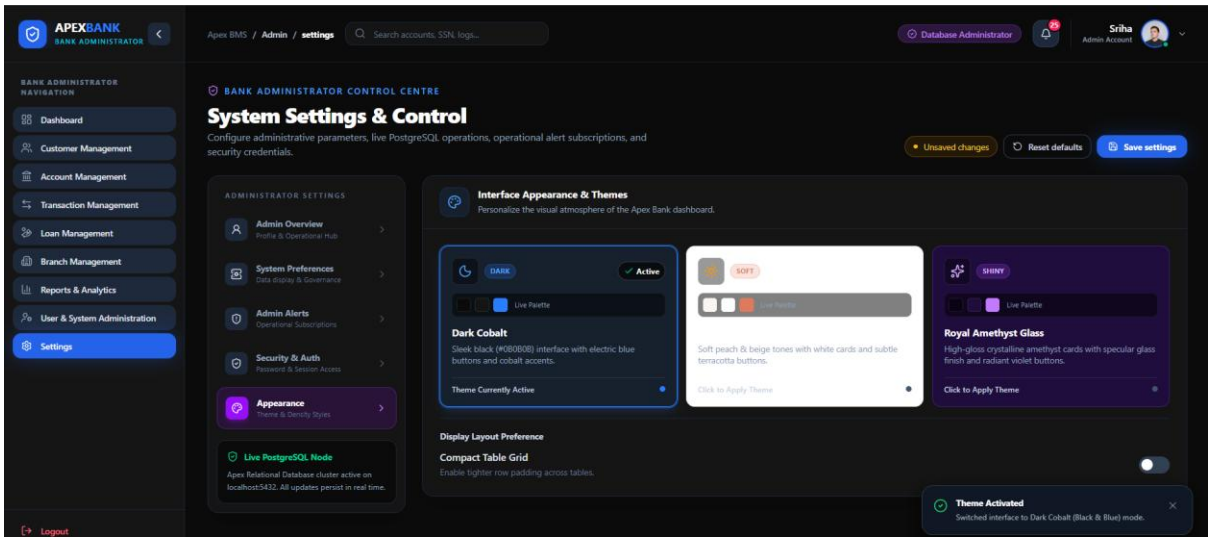


Figure 15: Admin Settings

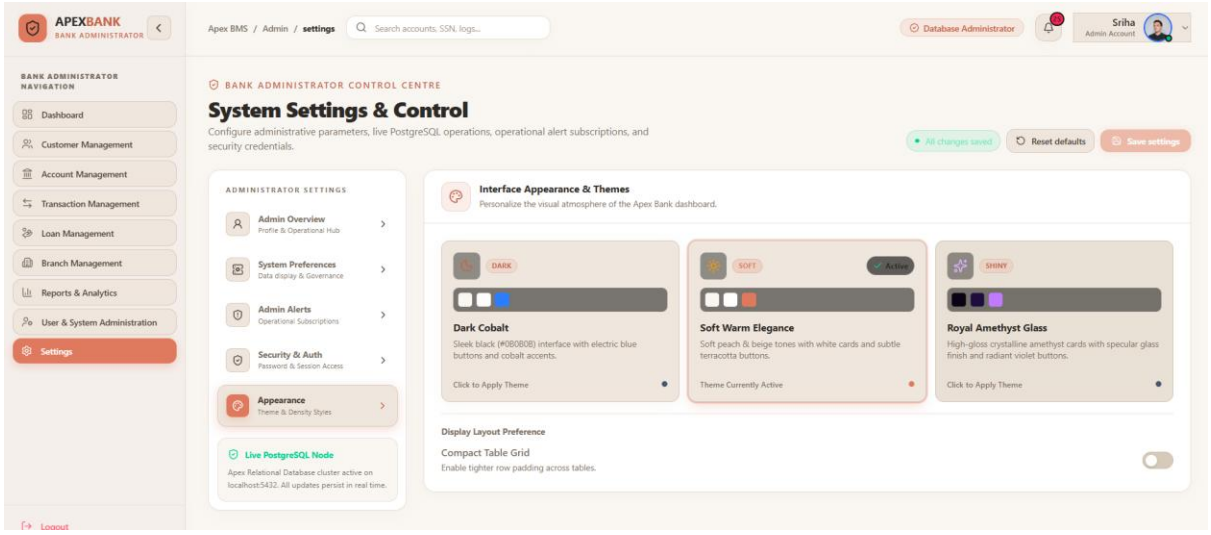


Figure 16: Soft theme [Alternative]

10. Challenges Faced & Solutions

Challenge	Solution
Every account needs a branch (Account.BranchID is NOT NULL), so account creation failed on an empty database.	A default branch (Apex Main Branch, IFSC APEX0000001) is inserted with INSERT ... ON CONFLICT (IFSCCode) DO NOTHING, so the script can be re-run safely.
The first default-branch insert left out City, which is NOT NULL, and was rejected.	The insert was corrected to supply Address and City along with the name and IFSC code.
Loan.EmployeeID was NOT NULL, which blocked customers from applying for a loan because no employee is known at application time.	ALTER TABLE Loan ALTER COLUMN EmployeeID DROP NOT NULL; the processing employee is assigned later (test F9). The ER diagram and cardinality table were updated to show partial participation.
USER is a reserved word in PostgreSQL; an unquoted SELECT * FROM User returns the current database user instead of the table.	The table is created and queried as “User” with double quotation marks everywhere, including every foreign key.
Several patch scripts using CREATE TABLE IF NOT EXISTS defined different columns, types and defaults (for example a LoanStatus default of Pending that is not in the CHECK list).	The schema was consolidated into one script that starts with DROP TABLE IF EXISTS ... CASCADE and uses the default Applied, which is in the CHECK list (test T11).
A failed or partial update could leave the ledger and the balance inconsistent, and an overdraft could make a balance negative.	Ledger insert and balance update run in one transaction, and CHECK (Balance >= 0) makes an overdraft roll everything back (tests F6 and T4).
The first normalization example used a single-attribute key, so 2NF was trivially satisfied and showed nothing.	The statement relation was redesigned with the composite key (AccountID, TransactionSeqNo), which exposes genuine 1NF, 2NF and 3NF violations (Section 5.7).
The Role stored in User can disagree with the subclass table that holds the user.	Role is restricted by a CHECK constraint and kept consistent by application logic; a composite foreign key or trigger is listed as future work.

11. Conclusion & Future Enhancements

11.1 Conclusion

The Apex Bank Management System shows how a well-designed relational database supports a real banking application. The ten-table schema is normalized to BCNF and protected by primary keys, foreign keys and UNIQUE, CHECK, NOT NULL and DEFAULT constraints. Specialising User into Customer, Employee and Admin keeps role-specific data separate, and treating Transaction as an append-only weak entity of Account preserves a permanent, replayable history of every deposit, withdrawal and transfer. Running each money movement as one database transaction keeps the ledger and the balances consistent, and the CHECK constraint on the balance stops overdrafts. The 28 test cases, all of which passed, confirm that the objectives of the project were achieved.

11.2 Future enhancements

- **Ledger trigger:** a PL/pgSQL trigger that rejects UPDATE and DELETE on Transaction and fills BalanceAfter automatically.
- **Nominee and loan triggers:** enforce that nominee shares total 100 and that LoanStatus only moves in the allowed order.
- **Role consistency:** a composite foreign key or trigger so that User.Role always matches the subclass row.
- **Stricter constraints:** add NOT NULL to Amount, PrincipalAmount, InterestRate and TenureMonths, because a CHECK constraint alone lets NULL values through.
- **Joint accounts:** replace the single owner with an Account–Customer linking table.
- **Interest and EMI:** interest accrual for Savings and Fixed Deposit accounts, maturity handling, and a repayment schedule table for loans.
- **Employee portal:** a separate employee tab so that staff can process loans directly.
- **Security:** two-factor authentication, session expiry, an audit log of administrator actions, and removal of CVV storage in line with PCI-DSS.
- **Performance:** indexes on foreign-key columns such as Account.CustomerID and Loan.CustomerID, and cloud hosting with managed PostgreSQL.

References

[1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed., Pearson, 2016.

[2] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed., McGraw-Hill, 2019.

[3] PostgreSQL Global Development Group, PostgreSQL Documentation (SQL commands, constraints and transactions). https://www.postgresql.org/docs/

[4] pgAdmin documentation (ERD tool and query tool). https://www.pgadmin.org/docs/

[5] MDN Web Docs: HTML, CSS, JavaScript and the Fetch API. https://developer.mozilla.org/

[6] Vercel documentation. https://vercel.com/docs

[7] Course material, 23CSE202 Database Management Systems, Amrita School of Computing, Amritapuri Campus.